20.3.26

Freitag, 20. März 2026 21:21

Heute habe ich angefangen ein Black jack Programm zu schreiben zum üben. Bis jetzt kann es eine zufällige Zahl nehmen und diese zu einer Karte machen und zu einem Wert zuordnen. Als nächstes will ich, dass das Programm erkennen kann ob jemand black jack hat oder draussen ist. Danach will ich den Dealer Algortihmus machen und zum schluss werde ich noch ein Wincheck system machen

22.03.26

Sonntag, 22. März 2026 22:10

Das ist mein heutiger Fortschritt. Ich habe vor allem am allgemeinen Programm viele Logikfehler und Bugs behoben. Ein Beispiel davon ist, dass das Programm ausgeteilte Karten nicht richtig aus dem deck Vector gelöscht hat, da ich dieses nicht von 0 bis 51 sondern von 1 bis 52 nummerieren liess. Dies ist zwar einfacher für mich in vielen Fällen, jedoch hat es hier zu problemen geführt, da der Index 5 nicht der Karte 5 entspricht, sondern der Karte 4 aufgrund davon dass der erste Wert den Index 0 hat. Dazu habe ich ein System gemacht, das die Karten eines Spielers summiert und habe allgemein den Spielablauf aufgebaut. Nun muss ich noch ein System machen, das Busts/Siege erkennt sowie den Dealer algorithmus. Dazu muss ich noch die zweite Form des Asses berücksichtigen

24.03.26

Dienstag, 24. März 2026 17:35

Mein Blackjack Programm funktioniert nun einigermassen. Der Dealer Algorithmus funktioniert und ich habe auch einen Weg gefunden um Asse als 11 oder 1 zählen zu lassen, je nach dem was vorteilhaft ist. Ich habe den Code in Chatgpt getan, um nach Feedback zu fragen. Ein paar problematische Aspekte meines Codes sind dabei:

- Die Zufallsgeneration erfolgt jedesmal wieder neu.
- Die Indexes im Deck sind als Wert von 1-52 sortiert, das ist gefährlich, da Index 5 nun für karte 6 steht.
- Das Deck wird nicht richtig gemischelt.

Dazu wäre ein Geldsystem vorteilhaft. Ausserdem hat er mir kleine, leicht behebbare Fehler aufgezeigt, welche ich schnell beheben konnte.

15.04.26

Mittwoch, 15. April 2026 19:04

Ich habe versucht ein minesweeper game zu machen. Es funktioniert soweit, mit dem einzigen Problem, dass bei dem Aufdecken vieler Felder es probleme hat nach oben. Dies liegt daran, dass der for-loop nicht rückwärts gehen kann. Somit müsste ich ihn laufen lassen, bis die Expansion von Leeren Feldern vorbei wäre, jedoch habe ich noch keinen Weg für das gefunden. Dazu muss ich noch den Spieler 'Flaggen' platzieren lassen.

16.04.26

Donnerstag, 16. April 2026 16:32

Heute konnte ich alle der grössten Probleme lösen. Der Grund, warum der Code eine so grosse Zeile hat ist, da dies eine riesige Bedingung ist für das prüfen, ob Eine 9 ein '*' als Nachbar hat ohne, dass die Funktion out of bounds geht. Als mein nächstes Projekt möchte ich ein Programm bauen, das ein Labyrinth generiert und anschliessend löst.

17.04.26

Samstag, 18. April 2026 00:05

Das ist mein bisheriger Fortschritt am Labyrinth Programm. Eine grosse Lektion, die ich von minesweeper mitgenommen habe ist die eines beliebig grossen Feldes. Indem ich nämlich alle von der Grösse des Feldes abhängige Aktionen mit einer Variabel ausgetauscht habe, kann ich die grösse des Feldes spontan ändern. Jedoch muss ich nun Vektoren benutzen statt arrays, da diese dynamisch gross sein können. Bis lang funktioniert alles im Programm bis auf den Expansion teil. Meine Idee war es, nachdem ein Anfangsfeld per Zufall gewählt wurde, würde von dort aus ein Hauptpfad entstehen. Dieser würde vom start aus per Zufall eine Route nehmen, bis es irgendwo an den Rand kommt, wo dann das Ende sein wird. Allerdings funktioniert etwas in der Funktion nicht. Bislang konnte ich noch nicht herausfinden, was es ist. Nachdem ich jedoch dieses Problem behebe, wird meine Idee sein, an zufälligen Stellen im Hauptpfad, Seitenpfade einzubauen. Diese hätten dann eine zufällige Länge, können jedoch weder in den Hauptpfad noch an den Rand zurück, um den einen klaren Weg zu erhalten.

18.04.26

Samstag, 18. April 2026 19:00

Ich konnte das Problem der Expansion Funktion lösen. Jedoch weiss ich nicht, was gestern falsch lief. Ich habe nämlich ab einem gewissen Punkt einfach aufgegeben und habe die gesamte Expansion Funktion nochmals geschrieben. An dem Punkt konnte mein Programm sich einen Weg buddeln und an einem zufälligen Ort wieder rauskommen. Jedoch war überraschenderweise das markieren des Ende mit einem '@' das schwierigste. Ich dachte zuerst, dass ich einfach bei der edger Funktion einfach einfügen könnte, dass nachdem es ein 'O' am Rand gefunden hat, würde es dieses mit einem '@' ersetzen und danach false zurückgeben, was die rekursiven Funktionen auflösen würde. Jedoch war mir nicht bewusst, dass jede einzelnen rekursive Funktion einzeln aufgelöst werden müsste. Somit könnte zwar die letze beendet werden, jedoch würde die nächste wieder die Funktion neu starten, da die edger Funktion kein 'O' mehr am Rand sieht. Dazu hatte ich nichts, das die Expansion Funktion daran hindert, in eine Sackgasse zu expandieren und das Programm in einen unendlichen loop zu stürzen. Dies war mir erst nach viel Frust und einem verzweifelten Gespräch mit ChatGPT bewusst. Somit musste ich in der Bedingung für den edger noch ein Einschub machen, dass es false zurückgeben würde, sobald zwei 'O' oder '@' am Rand gefunden wurden. Ansonsten hatte ich noch ein paar Probleme mit den bounds, da ich ständig unsaubere Bedingungen geschrieben habe, welche die aus dem Defintionsbereichen meiner Vektoren gingen und somit das programm crashten. Warscheinlich war genau ein solcher out of bounds error das Problem an meiner ursprünglichen Expansion Funktion, allerdings hatte ich an dem Punkt schon zu viel Schwachsinn gemacht, dass ich den Fehler finden konnte. Das Problem mit den Sackgassen hingegen habe ich vorerst in die Ecke gestellt, indem ich gesagt habe, es sollte das Programm einfach beenden, sobald er zehn mal keinen Weg findet. Als nächstes würde ich noch gerne den Algorithmus zwingen, auf einer neuen Wand zu enden, als dass er gestartet ist. Dazu möchte ich auch einen saubereren Weg finden, um Sackgassen zu vermeiden. Spontan denke ich, dass ich das Programm einfach neu starten könnte, sobald eine Sackgasse erreicht wurde. Dazu würde ich noch gerne 'Unterpfade' einbauen. Diese würden so funktioneren, dass es ein zufälliges 'O' auswählen würde und von dort aus nochmals einen Pfad macht. Dieser soll jedoch weder in den ursprünglichen Pfad noch an den Rand gehen. Dies ist, damit ein Lösungspfad erhalten bleiben kann. Dazu wäre es auch schön, wenn ich es verhindern könntte, dass die Expansion Funktion keine Flächen macht sondern einen schönen Pfad. Jedoch könnte dies etwas schwierig werden. Mein Endziel bei diesem mini-Projekt ist es, dass ich einen Labyrinth Algorithmus einbauen kann, der das Labyrinth dann selber löst.

26.04.26

Mittwoch, 22. April 2026 15:04

Ich habe heute versucht, den Labyrinth Algorithmus etwas schöner zu machen. Dafür habe ich mich bei Wikipedia ein wenig informiert und bin dann auf die Idee gekommen, dass ich das Programm nur dann expandieren lassen würde, wenn es am neuen Ort an keine 'O' Grenzt. Jedoch ist mir dies bis jetzt noch nicht gelungen. Dennoch habe ich eine einigermassen funktionierende Funktion, die mir sagt, ob ein Feld an ein O grenzt. Dabei ist noch wichtig die Herkunftsrichtung der Expansion zu beachten. Jedoch scheint etwas immernoch nicht zu gehen. Warscheinlich liegt es daran, dass das Programm nirgendswo hin expandieren kann. Ich werde es jedoch noch probieren zu lösen.

https://en.wikipedia.org/wiki/Maze_generation_algorithm

28.04.26

Dienstag, 28. April 2026 17:00

Ich habe es geschafft, das Programm zu zwingen, ein schönes Labyrinth zu machen. Ich habe dies getan, indem ich der Expansionsfunktion gesagt habe, sie dürfe nicht auf ein Feld expandieren, das an ein 'O' grenzt, das nicht das 'O' ist, von dem es gerade gekommen ist. Ich habe ein paar versuche gebraucht, bis ich es geschafft habe aber nun geht es. Aktuell möchte ich das Programm noch zwingen, an einen anderen Rand zu gehen. Das sollte die finisherfunktion regeln. Die Funktion scheint zu gehen, jedoch geht irgendetwas mit der Logik noch nicht. Somit werde ich als nächstes dies beenden. Danach will ich noch alternative Zweige einbauen, sodass es auch wirklich zu einem Labyrinth wird. Am Ende würde ich noch gerne einen Algorithmus einbauen, der das Labyrinth nehmen kann und auch lösen kann.

30.04.26

Donnerstag, 30. April 2026 01:49

Ich konnte das Problem mit den Rändern beheben. Das Problem lag einerseits in der Logik bei der wiederholten Generation von zufälligen Richtungen. Andererseits jedoch lag es auch an der Funktion selbst. Ich habe nämlich einen switch benutzt, um die einzelnen Richtungen darzustellen: Jedoch habe ich vergessen aus dem switch auszubrechen, was man unbedingt tun muss. Dazu habe ich einen Logikfehler gemacht. Ich hatte nämlich einetseits eine Bedingung, die rüchzüge und überschreiben vermied und die finisher Funktoin in eine and Bedingung gesetzt. Nun ist dies jedoch falsch, da es die Generation dann nur wiederholt, wenn bei beiden Unterbedingungen etwas falsch ist. Jedoch sollte es die Generation wiederholen, wenn auch nur bei einem der beiden Bedingungen ein Fehler auftritt. Somit musste ich sie mit eine or verbinden, was nun klappt. Jedoch ist mir während dem testen aufgefallen, dass das @ immernoch in der Ecke spawnen kann, was es nicht sollte. Dies werde ich noch beheben, wonach ich schlafen gehen werde. Ich eiss nicht warum ich jetzt noch am programmieren bin Es ist bald 2. Ich bin auf eine Art süchtig unf kann nicht mehr aufhören bis ich die bugs in ihre Einzelteile zersetzt habe und sie mich nicht mehr plagen. Ich hoffe ich denke daran den letzten Teil rauszunehmen. Sonst, tut es mir leid.

Update: Das Ecken Problem Ist weg. Gute nacht

01.05.26

Freitag, 1. Mai 2026 18:37

Heute habe ich versucht verzwigungen im Hauptpfad zu machen. Jedoch macht das Programm bis jetzt nur ein 'O' an eine Stelle hin und bricht dann ab. Meine Vermutung ist, dass es etwas mit der Bedingung der while Schleife der Recursion zu tun hat. Mein Ziel wäre es, dass das Programm an zufälligen 'O's eine zufällig langen, legalen Substreifen macht. Zudem würde ich gerne noch das Generationsprogramm zwingen, eine gewisse mindestlänge einzuhalten für den Hauptpfad, da dieser oftmals bei grösseren Werten von A nur am Rand vorbeischleicht. Eine Idee wäre es, dem Generationsalgorithmus, eine Mindestlänge an 'O's zu geben. Bis es an den Rand gehen kann. Eine andere Option wäre es, den Hauptpfad zu zwingen durch das Zentrum zu gehen oder an der gegenüberliegenden Seite zu enden.

13.05.26

Mittwoch, 13. Mai 2026 20:39

Heute habe ich weitergemacht bei den verzweigungen. Zwar habe ich es geschafft, dass es verzweigungen macht, allerdings bin ich noch nicht ganz zufrieden damit, wie diese gemacht werden. Aktuell kann ich dem Programm noch sagen, wie lange diese sein sollten, jedoch muss ich um dies zu tun die 'visited' Liste löschen, sodass gewisse Felder theoretish zweimal mit 'O' beschrieben werden, könnten, was in der Realität einfach bedeuten würde, dass nichts passiert. Dies müsste ich definitiv beheben. Jedoch hatte ich noch eine andere Idee um das Problem zu lösen. Nämlich könnte ich schauen, ob es legale Expansionen in der Stellung gibt und solange es diese gibt lasse ich die Expansionsfunktion laufen. Dies würde dann auch zu einem kompletten Labyrinth führen. Jedoch habe ich da noch ein paar Probleme.

26.05.26

Dienstag, 26. Mai 2026 21:36

Ich habe ziemlich guten fortschritt gemacht. Mein Programm kann nun erkennen, welche 'O' legale Expansionen haben und wohin. Im nachhinein hätte ich dies schon viel früher machen sollen, allerdings ist mir dies erst jetzt eingefallen. Allerdings hat das Programm noch das Problem, dass irgendetwas beim überspringen unexpandierbarer Felder falsch geht. Falls ich dies lösen kann denke ich, dass ich praktisch fertig bin mit der Generation des Labyrinths.

29.05.26

Freitag, 29. Mai 2026 19:12

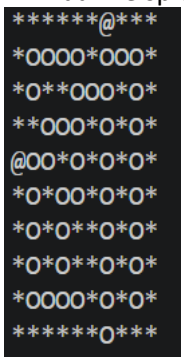
Eigentlich habe ich sehr guten Fortschritt gemacht. Allerdings hat das Programm aus dem nichts aufgehört zu funktionieren. Ich habe keine Ahnung was das Problem ist. Ich werde versuchen es zu finden, allerdings denke ich, dass ich vielleicht aus einer vergangenen Version wieder neu beginnen muss

30.5.26

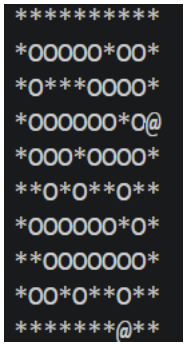
Samstag, 30. Mai 2026 14:36

Ich habe mit der vergangenen Version weitergemacht, da ich das Problem in der neuen nicht fand. Als ich allerdings meine Schritte wiederholt habe, habe ich dann den Fehler gefunden. Das Problem lag darin, dass mein legalmaze nicht richtig geupdated war am anfang und somit das program dacht, dass es keine legale Züge gab. Jedoch konnte ich dies beheben. Mein Programm ist mittlerweile soweit, dass es fast alle legalen Optionen ausschöpft. Jedoch hat es wieder angefangen, illegale Expansionen vohrzunehen.. Ich werde herausfinden müssen, woran das liegt.

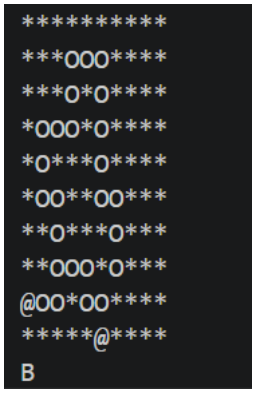
Ein Paar Beispiele:



Wie man sehen kann ist die viertletzte Reihe etwas problematisch. Auf der einen Seite könnte es nach links noch zum Teil expandieren. Auf der anderen Seite hat es zu weit expandiert und ist an den Rand gekommen, was es nicht darf.



In diesem Beispiel gibt es zwar keine legale Expanionen mehr, jedoch wurden auch illegale Expansionen durchgeführt, wie man in der 3-4 letzten Spalte sehen kann, wo viele 'O' mehr als zwei andere berühren, was nicht legal sein sollte.



Jedoch gibt es immernoch solche Beispiele, wo zwar keine 'O' illegal stehen, allerdings gibt es noch viele legale Optionen, die noch nicht ausgeschöpft wurden.

31.05.26

Sonntag, 31. Mai 2026 20:11

Mein Labyrinthprogramm kann sich fast komplett ausdehnen. Jedoch bricht es immer kurz vor Ende unerklärlich ab. Allerdings macht es keine falsche Expansionen mehr. Die falschen Expansionen lagen daran, dass in der legal_expansion Funktion ich einerseits nicht überprüft habe, ob der loop out of bounds geht. Andererseits habe ich alte 'O's, die einst legale Expansionen hatten, nie überschrieben habe. Der Bug, dass das Programm aufhört, liegt ziemlich sicher daran, dass es irgendwo in der Expansion Funktion oder sonst wo hängen bleibt. Dies liegt daran, dass nichts nach dem zweiten Expansionsloop ausgeführt wird.

1.06.26

Montag, 1. Juni 2026 20:47

Ich habe den Bug gefunden, der das Programm zum hängen bleiben zwang. Ich habe nämlich vergessen, die schon besuchten 'O' aus der besuchten Liste zu löschen. Mein Programm geht jetzt zwar, allerdings baut es keine schöne Labyrinthe. Das Programm hat nämlich eine gewisse Affinität dafür, lange Reihen zu bilden. Dies liegt vermutlich an der Tatsache, dass ich keine gute random Funktion habe.